

Certification of Imitation Controllers Using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann

Date: August 4, 2026, Date of submission: 16. August 2026

1. Review: Prof. Dr.-Ing. Rolf Findeisen

2. Review: Hendrik Alsmeier M.Sc.

Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

CCPS

Control and Cyber-Physical Systems

Electrical Engineering and
Information Technology
Department

Certification of Imitation Controllers Using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.

Date: August 4, 2026

Date of submission: 16. August 2026

Darmstadt

Technische Universität Darmstadt
Institut für Automatisierungstechnik und Mechatronik
Fachgebiet Control and Cyber-Physical Systems
Prof. Dr.-Ing. Rolf Findeisen

Contents

1	Abstract	1
2	Introduction	2
3	Optimal Control and Lyapunov Stability Theory	3
3.1	Discrete-Time Dynamical Systems and Stability	3
3.1.1	Lyapunov Stability	3
3.2	Model Predictive Control and Stability	4
3.2.1	Problem Formulation	4
3.2.2	Terminal Ingredients for Stability	4
3.2.3	Recursive Feasibility and Closed-Loop Stability	5
4	Foundations of Neural Control Policies	7
4.1	Artificial Neural Networks	7
4.1.1	Multi-Layer Perceptrons	7
4.1.2	Optimization and Training	9
4.2	Adversarial Training and Falsification	9
5	Formal Verification of Neural Control Policies	10
5.1	Mixed-Integer Linear Programming	10
5.2	Branch and Bound	10
5.3	$\alpha\beta$ -CROWN	10
6	Learning Lyapunov-Stable Imitation Controllers	11
6.1	Neural Policy Approximation and Imitation Learning	11
6.2	Lyapunov Learning for Stability Certification	13
6.2.1	Lyapunov Network Architecture	13
6.2.2	Loss Formulation	13
6.2.3	Falsification-Guided Training	18
6.2.4	ρ -Sublevel Estimation	20
6.3	Formal Verification Methodology	21
7	Experimental Evaluation and Certification Results	24
7.1	System Modeling and Problem Formulation	24
7.1.1	Double Integrator	24
7.1.2	Cartpole	24
7.2	Linear System Results: Double Integrator	25
7.3	Nonlinear System Results: Cartpole	25

8 Discussion	27
8.1 Qualitative Discussion of Results	27
8.2 Challenges with Nonlinear Dynamics	27
8.3 Scalability and Limitations	27
9 Conclusion	28
A Appendix	29
Bibliography	31

List of Variables and Symbols

Symbol	Description
<i>System Dynamics & Control</i>	
n_x	State space dimension
n_u	Control input space dimension
$\mathcal{X}$	Admissible state set
$\mathcal{U}$	Admissible input set
$\underline{x}$	Lower state bound
$\bar{x}$	Upper state bound
$\underline{u}$	Lower control input bound
$\bar{u}$	Upper control input bound
T_s	Sampling time
k	Discrete time step index
$x(t)$	Continuous-time state vector
x_k	Discrete-time state vector
$u(t)$	Continuous-time control input vector
u_k	Discrete-time control input vector
x^*	System equilibrium point
u^*	Input reference at x^*
x_{k+1}	Discrete-time successor state
x^+	One-step successor state
$\pi(x)$	State-feedback policy
$f_c(x, u)$	Continuous-time system dynamics
$f(x_k, u_k)$	Discrete-time system dynamics
<i>Optimal Control & Model Predictive Control (MPC)</i>	
N	Prediction horizon
$\ell(x, u)$	Stage cost function
$V_f(x)$	Terminal cost function
$\mathcal{X}_f$	Terminal constraint set
Q	State weighting matrix
R	Input weighting matrix
u_0^*	Optimal control input
$V_N^*(x)$	Optimal value function
A	Linearized system state matrix
B	Linearized system input matrix
K	LQR feedback gain matrix

Symbol	Description
P	DARE solution matrix
α_N	Relaxed Lyapunov decrease factor
$\mathcal{X}_N$	Feasible initial state set
<i>Neural Network</i>	
L	Total layer count
l	Layer index
$a^{(l)}$	Layer activation vector
$z^{(l)}$	Layer pre-activation vector
$W^{(l)}$	Layer weight matrix
$b^{(l)}$	Layer bias vector
$\sigma^{(l)}(\cdot)$	Non-linear activation function
θ	Trainable network parameters
<i>Imitation Learning</i>	
$\mathcal{X}_\pi$	Imitation learning state space
$\underline{x}_\pi$	Admissible state lower bound
$\bar{x}_\pi$	Admissible state upper bound
$\mathcal{D}_\pi$	Imitation learning dataset
$N_{\mathcal{D}}$	Dataset sample count
$\mathcal{B}_\pi$	Training mini-batch
M_π	Mini-batch size
$\pi_0(x)$	Expert MPC policy
$\pi_\theta(x)$	Learned imitation policy
$h_\theta(x)$	Unconstrained neural network output
$\text{clamp}(\cdot)$	Control input clamping function
S_x	State scaling matrix
S_u	Input scaling matrix
x_{scale}	State scaling vector
u_{scale}	Input scaling vector
$d(x)$	Scaled equilibrium distance
$w_x(x)$	Behavioral cloning loss weight
w_{min}	Baseline loss weight
η	Decay rate for state-dependent weight
$\mathcal{L}_{\text{BC}}$	Behavioral cloning loss
$\mathcal{L}_f$	Dynamics-aware constraint loss
ω_{BC}	Behavioral cloning loss weight
ω_f	Dynamics-aware constraint loss weight
$\mathcal{L}_\pi$	Total imitation policy loss
<i>Neural Lyapunov Learning & Falsification</i>	
$\pi_V(x)$	Co-trained imitation policy
$V_\theta(x)$	Neural Lyapunov candidate
ε	Regularization constant
R_θ	Learnable transformation matrix

Symbol	Description
ρ	Target Lyapunov sublevel value
$\mathcal{V}_\rho$	Certified sublevel set
$\mathcal{X}_V$	Lyapunov training domain
$\rho_{\min}$	Minimum sublevel target bound
$\pi_V(x)$	Co-trained policy
e_V	Lyapunov decrease violation
κ	Target decrease rate
ε_{rel}	Numerical stabilization constant
$\mathcal{L}_{\text{dec}}$	Relative Lyapunov decrease loss
S_{range}	Domain range scaling matrix
$\mathcal{L}_{\text{inv}}$	Relative state-invariance loss
$\mathcal{L}_{\text{cond}}$	Combined condition violation loss
w_ρ	Soft sublevel gating weight
s	Gate sharpness parameter
$\mathcal{L}_{\rho\text{-gated}}$	Soft-gated condition loss
$\mathcal{B}_{\text{ROA}}$	Shell sample set
ξ	Inner boundary shell factor
$\mathcal{L}_{\text{ROA}}$	Region of Attraction expansion loss
$e_{V,\text{lower}}$	Signed Lyapunov decrease lower bound
$\mathcal{M}_L$	Region LIRPA condition violation
w_L	LIRPA region weight
α_L	LIRPA weight decay rate
N_L	Region count
$\mathcal{L}_L$	LIRPA loss
ℓ_V	Mean Lyapunov log-value
$\ell_{V,0}$	Initial mean Lyapunov log-value
$\mathcal{L}_V$	Lyapunov magnitude scaling loss
$\mathcal{L}_\pi$	Policy scaling loss
$\ell_{R,0}$	Initial matrix R Frobenius norm
$\mathcal{L}_R$	Matrix R regularization loss
$\mathcal{L}_{\text{total}}$	Total combined Lyapunov loss
ω_{cond}	Soft-gated condition loss weight
ω_{ROA}	Region of Attraction expansion loss weight
ω_L	LIRPA loss weight
ω_V	Lyapunov magnitude scaling loss weight
$\omega_{S\pi}$	Policy scaling loss weight
ω_R	Matrix R regularization loss weight
$\mathcal{C}$	Counterexample pool
$\mathcal{S}$	Explorative pool
x_{adv}	Counterexample state
J_{PGD}	PGD objective
τ	Counterexample age
τ_{max}	Maximum lifespan
m	Sublevel margin
γ	Overestimation factor

Symbol	Description
$\tilde{\rho}$	Estimated ROA boundary minimum
β	EMA decay rate
<i>Formal Verification & Certification</i>	
$\mathcal{B}_C$	Formal verification domain
$\Phi(x; \rho)$	Verification specification formula
Φ_{sub}	Sublevel set exclusion predicate
Φ_{pos}	Positive definiteness predicate
Φ_{inv}	Forward invariance predicate
Φ_{dec}	Strict Lyapunov decrease predicate
$\mathcal{B}_{\text{hole}}$	Equilibrium exclusion region
ρ^*	Maximal verified sublevel value



1 Abstract



2 Introduction

3 Optimal Control and Lyapunov Stability Theory

3.1 Discrete-Time Dynamical Systems and Stability

Implementing optimal control schemes on digital hardware requires a discrete-time representation. Consequently, the continuous-time dynamics

$$\dot{x}(t) = f_c(x(t), u(t)) \quad (3.1)$$

must be discretized, where $x(t) \in \mathbb{R}^{n_x}$ and $u(t) \in \mathbb{R}^{n_u}$ denote the continuous-time state and input vectors, respectively, with state space dimension $n_x \in \mathbb{N}$ and input dimension $n_u \in \mathbb{N}$. In this work, the corresponding discrete-time system

$$x_{k+1} = f(x_k, u_k) \quad (3.2)$$

is obtained by approximating the continuous dynamics using a numerical integration scheme with a constant sampling time $T_s > 0$. Here, $k \in \mathbb{N}_0$ denotes the discrete time step index, such that $x_k := x(kT_s) \in \mathbb{R}^{n_x}$ and $u_k := u(kT_s) \in \mathbb{R}^{n_u}$ represent the state and control input at time step k . The discrete-time function $f : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \mathbb{R}^{n_x}$ is evaluated using a numerical integration scheme, such as the forward Euler or a fourth-order Runge-Kutta (RK4) method [1].

To account for physical and operational limits, the system states and control inputs are restricted to compact constraint sets $\mathcal{X} := [\underline{x}, \bar{x}] \subseteq \mathbb{R}^{n_x}$ and $\mathcal{U} := [\underline{u}, \bar{u}] \subseteq \mathbb{R}^{n_u}$. Furthermore, without loss of generality, any equilibrium (x^*, u^*) can be shifted to the origin via deviation variables $\tilde{x} := x - x^*$ and $\tilde{u} := u - u^*$. To keep notation concise, the tildes are omitted and x and u are used directly for the shifted variables, assuming $(x^*, u^*) = (\mathbf{0}, \mathbf{0})$ throughout this chapter.

3.1.1 Lyapunov Stability

Consider the discrete-time closed-loop system under a state-feedback control policy $\pi : \mathcal{X} \rightarrow \mathcal{U}$ with the origin $x = \mathbf{0}$ as an equilibrium point, satisfying $\pi(\mathbf{0}) = \mathbf{0}$ and $f(\mathbf{0}, \mathbf{0}) = \mathbf{0}$. Assuming the state constraint set $\mathcal{X}$ is positively invariant under π , the origin is locally asymptotically stable in $\mathcal{X}$ if there exists a positive definite function $V : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}$ satisfying [2, Theorem 2.13]:

$$\begin{aligned} V(\mathbf{0}) &= 0, \\ V(x) &> 0 \quad \forall x \in \mathcal{X} \setminus \{\mathbf{0}\}, \\ V(f(x, \pi(x))) - V(x) &< 0 \quad \forall x \in \mathcal{X} \setminus \{\mathbf{0}\}. \end{aligned} \quad (3.3)$$

The strict decrease condition guarantees trajectory convergence to the origin for all initial states x_0 within a certified domain.

look up if all assumptions are named

3.2 Model Predictive Control and Stability

This section summarizes the model predictive control (MPC) framework and the conditions required to guarantee closed-loop stability in finite-horizon optimal control setups.

3.2.1 Problem Formulation

For the nominal MPC setup considered in this work, the following finite-horizon optimal control problem is solved at each time step k given the current system state x_k [2, Sec. 2.2, 3, Sec. 3.3]:

$$\begin{aligned}
 \min_{\{x_i\}_{i=0}^N, \{u_i\}_{i=0}^{N-1}} \quad & \sum_{i=0}^{N-1} \ell(x_i, u_i) + V_f(x_N) \\
 \text{s.t.} \quad & x_{i+1} = f(x_i, u_i), \quad i = 0, \dots, N-1, \\
 & x_i \in \mathcal{X}, \quad i = 0, \dots, N-1, \\
 & u_i \in \mathcal{U}, \quad i = 0, \dots, N-1, \\
 & x_N \in \mathcal{X}_f, \\
 & x_0 = x_k.
 \end{aligned} \tag{3.4}$$

Here, $N \in \mathbb{N}$ denotes the prediction horizon, $\ell : \mathcal{X} \times \mathcal{U} \rightarrow \mathbb{R}_{\geq 0}$ is the stage cost function, and $V_f : \mathcal{X}_f \rightarrow \mathbb{R}_{\geq 0}$ represents the terminal cost function defined over the terminal constraint set $\mathcal{X}_f \subseteq \mathcal{X}$. To distinguish predicted states and inputs from the actual closed-loop state x_k and control input u_k , the sequences $\{x_i\}_{i=0}^N$ and $\{u_i\}_{i=0}^{N-1}$ serve as optimization variables local to time step k , constrained by the sets $\mathcal{X}$ and $\mathcal{U}$ [3, Sec. 3.2].

For this work, the stage cost is restricted to a quadratic cost function with positive definite weighting matrices $Q \succ 0$ and $R \succ 0$, which is defined as follows [2, Sec. 1.3]:

$$\ell(x_i, u_i) = x_i^\top Q x_i + u_i^\top R u_i, \tag{3.5}$$

Following the receding horizon principle, only the first control input u_0^* from the optimal sequence is applied to the system, and the optimization is repeated at the next sampling instance [2, Sec. 2.2, 3, Sec. 3.1].

3.2.2 Terminal Ingredients for Stability

To guarantee closed-loop stability of a nominal MPC, the optimal control problem (3.4) incorporates specific terminal ingredients: the terminal cost function $V_f(x_N)$ and the terminal set $\mathcal{X}_f$ [2, Assumption 2.14, 3, Assumption 5.9].

The terminal set $\mathcal{X}_f$ is required to be positively invariant under a local control law $u = \pi(x)$, meaning that any trajectory originating in $\mathcal{X}_f$ remains inside the set for all future time steps while respecting input constraints [2, Def. 2.9]:

$$f(x, \pi(x)) \in \mathcal{X}_f \quad \text{and} \quad \pi_f(x) \in \mathcal{U}, \quad \forall x \in \mathcal{X}_f. \tag{3.6}$$

add contraction: $V_f(f(x, \pi_f(x))) - V_f(x) \leq -ell(x, \pi_f(x))$

The predictive mechanism is illustrated in Fig. 3.1. While the intermediate states $x_0, \dots, x_{N-1}$ are merely constrained to the safe state space $\mathcal{X}$, the terminal constraint enforces $x_N \in \mathcal{X}_f$. Let $V_N^*(x)$ denote the optimal value function of (3.4) for a state x . Combined with the terminal cost $V_f(x_N)$ acting as a local Lyapunov function in $\mathcal{X}_f$, these ingredients ensure that the optimal value function $V_N^*(x)$ serves as a Lyapunov function for the closed-loop system, guaranteeing asymptotic stability [2, Theorem 2.19].

For nonlinear systems, the terminal set $\mathcal{X}_f$ is typically computed using linearized system dynamics around the equilibrium point $(x, u) = (\mathbf{0}, \mathbf{0})$, yielding the linear discrete-time model:

$$x_{k+1} = Ax_k + Bu_k. \quad (3.7)$$

A widely used choice for these terminal ingredients is then derived from the Linear-Quadratic Regulator (LQR) [2, Eq. 1.18, 3, Eq. 5.24]. Here, the local control law $\pi(x_k)$ is chosen as a linear state-feedback policy $\pi(x_k) = -Kx_k$, where the optimal LQR gain K is given by

$$K = (R + B^\top PB)^{-1} B^\top PA. \quad (3.8)$$

Under the standard stabilizability and detectability assumptions, the matrix $P \succ 0$ is obtained as the unique stabilizing solution to the discrete-time Algebraic Riccati Equation (DARE) [2, Eq. 1.18, 3, Eq. 5.23]:

$$P = A^\top PA - A^\top PB (R + B^\top PB)^{-1} B^\top PA + Q. \quad (3.9)$$

This matrix parameterizes the quadratic terminal cost function $V_f(x_N) = x_N^\top P x_N$. Accordingly, the terminal set $\mathcal{X}_f$ is defined in this specific case as an ellipsoidal sub-level set of this cost function [2, Sec. 2.5.5]:

$$\mathcal{X}_f := \left\{ x \in \mathbb{R}^{n_x} \mid x^\top P x \leq \alpha \right\}. \quad (3.10)$$

Choosing the scaling parameter $\alpha > 0$ sufficiently small guarantees that $\mathcal{X}_f$ satisfies both the state and input constraints $\mathcal{X}$ and $\mathcal{U}$ while ensuring that the quadratic Lyapunov decrease dominates higher-order linearization errors of the nonlinear dynamics.

3.2.3 Recursive Feasibility and Closed-Loop Stability

more detail on recursive feasibility, smoother transition to closed loop stability.

Under the MPC policy $\pi_0(x_k) := u_0^*(x_k)$, the optimal value function $V_N^*(x_k)$ of (3.4) serves as a Lyapunov candidate over the feasible set $\mathcal{X}_N \subseteq \mathcal{X}$ [2, Sec. 2.4.2]. Using the terminal ingredients from Section 3.2.2, the recursive feasibility at time step $k+1$ is determined by using the remaining inputs of the optimal sequence computed at time step k and appending the terminal control action $\pi(x_N^*)$ [2, Sec. 2.4.2].

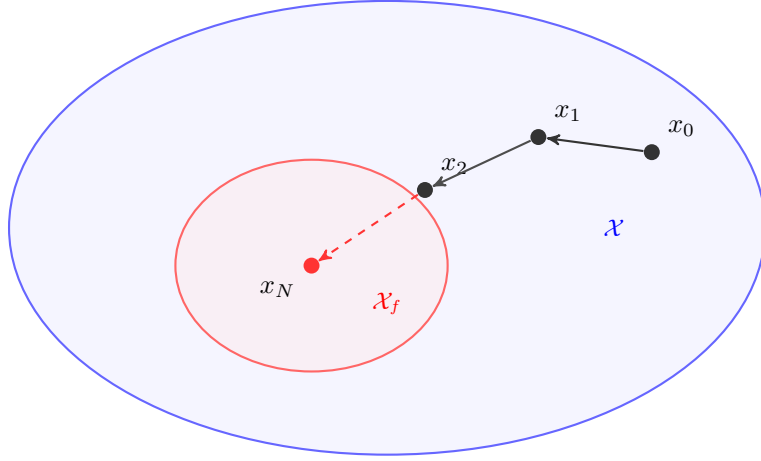


Figure 3.1: Comparison of the terminal set $\mathcal{X}_f$ (red) with the entire state space $\mathcal{X}$ (blue).

Consequently, V_N^* satisfies a decrease condition along closed-loop trajectories $x_{k+1} = f(x_k, \pi_0(x_k))$. Relaxing the standard stability conditions, this can be expressed with the relaxed Lyapunov decrease formulation [3, Theorem 4.11]:

$$V_N^*(x_{k+1}) \leq V_N^*(x_k) - \alpha_N \ell(x_k, \pi_0(x_k)), \quad \forall x_k \in \mathcal{X}_N, \quad (3.11)$$

where $\alpha_N \in (0, 1]$ denotes the relaxed decrease factor [3, Theorem 4.11]. Setting $\alpha_N = 1$ yields the standard nominal Lyapunov decrease

$$V_N^*(x_{k+1}) - V_N^*(x_k) \leq -\ell(x_k, \pi_0(x_k)), \quad (3.12)$$

guaranteeing asymptotic stability of the origin $\mathbf{0}$ for all initial states $x_0 \in \mathcal{X}_N$ [2, Assumption 2.14].

4 Foundations of Neural Control Policies

4.1 Artificial Neural Networks

In imitation learning, artificial neural networks (ANNs) approximate complex control laws offline to bypass the computational overhead of online model predictive control (MPC) evaluations.

4.1.1 Multi-Layer Perceptrons

Multi-layer perceptrons (MLPs) are fully connected feed-forward networks that map an input through $L - 1$ hidden layers to an output layer [4, Sec. 1.4, 5, Sec. 3.1]. For a state input x , forward propagation to the network prediction $\hat{y}$ is defined recursively for layers $l = 1, \dots, L$:

$$a^{(l)} = \sigma^{(l)}(z^{(l)}) \quad \text{with} \quad z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad \forall l = 1, \dots, L, \quad (4.1)$$

where $a^{(0)} := x$ is the input, $z^{(l)}$ is the pre-activation vector of layer l , and $a^{(L)} := \hat{y}$ is the network prediction. The matrix $W^{(l)}$ and vector $b^{(l)}$ denote the weights and biases of layer l , respectively, while $\sigma^{(l)}(\cdot)$ represents a non-linear activation function applied element-wise to the pre-activation vector $z^{(l)}$ [5, Sec. 3.2]. Without these non-linearities, the layers would simply be reduced to a single linear transformation, drastically limiting the model's ability to capture complexity [4, Sec. 1.5]. Common activation functions in neural control include the hyperbolic tangent (4.2a) and the rectified linear unit (ReLU) (4.2b), defined respectively as [5, Sec. 3.5].

$$\tanh z = \frac{e^{2z} - 1}{e^{2z} + 1} \quad (4.2a)$$

$$[z]_+ = \max(0, z) \quad (4.2b)$$

Their qualitative profiles are illustrated in Fig. 4.1.

Based on the universal approximation theorem, an MLP with at least one sufficiently sized hidden layer and non-linear activations can approximate any continuous multidimensional function on compact sets to arbitrary accuracy [5, Sec. 3.1]. The trainable parameters are represented by $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^L$. Figure 4.2 illustrates a representative architecture.

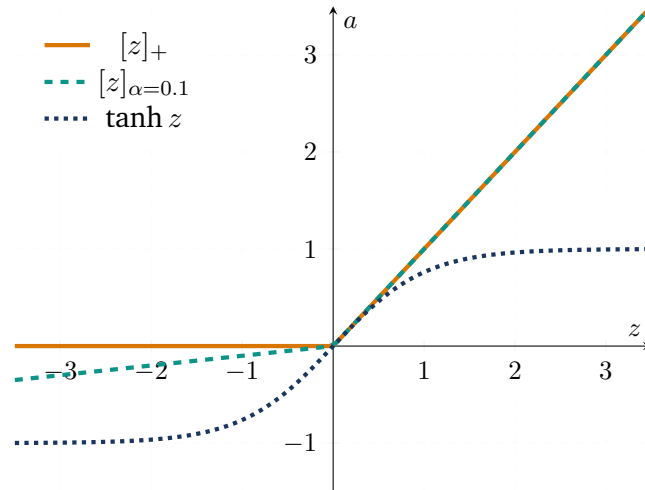


Figure 4.1: Standard activation functions for neural control policies.

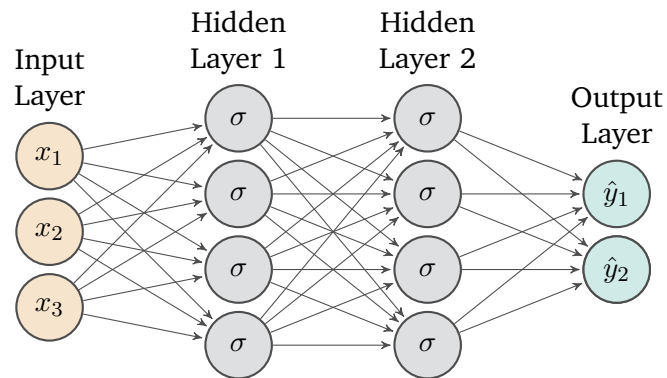


Figure 4.2: Architecture of an exemplary Multi-Layer Perceptron with three inputs, two outputs and two hidden layers with four neurons each. Illustration adapted from [5, Fig. 3.1]

4.1.2 Optimization and Training

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m \|\hat{y}_i - y_i^*\|_2^2. \quad (4.3)$$

dataset, mse, backprop, adam, mini batch, loss augmentation

4.2 Adversarial Training and Falsification

5 Formal Verification of Neural Control Policies

5.1 Mixed-Integer Linear Programming

5.2 Branch and Bound

5.3 $\alpha\beta$ -CROWN

6 Learning Lyapunov-Stable Imitation Controllers

This section specifies the data generation process, the neural policy architecture, and the training objective used to approximate the model predictive control (MPC) expert. Specifically, the MPC controller introduced in Section 3.2.1 serves as the expert policy generating the required state-input pairs. The neural policy is subsequently trained to reproduce the expert control actions over the considered state domain.

add lyapunov learning stuff

6.1 Neural Policy Approximation and Imitation Learning

Data Generation uses the MPC expert policy π_0 defined in Section 3.2.3. Closed-loop trajectories $x_{k+1} = f(x_k, \pi_0(x_k))$ are initialized from states sampled across the training set $\mathcal{X}_\pi : \{x \in \mathcal{X} \mid \underline{x}_\pi \leq x \leq \bar{x}_\pi\}$. Storing the visited states and their corresponding optimal control actions yields the dataset

$$\mathcal{D}_\pi = \left\{ (x^{(i)}, u^{(i)}) \right\}_{i=1}^{N_D}, \quad (6.1)$$

where $x^{(i)} \in \mathcal{X}_\pi$ and $u^{(i)} = \pi_0(x^{(i)})$.

move to 7

If the MPC problem becomes infeasible at any time during one trajectory, all training pairs associated with that trajectory are discarded.

To reduce the overrepresentations of regions with high sampling rates, near-duplicate state-action pairs are removed during preprocessing. The concatenated state-action vectors are normalized and mapped onto an adaptive voxel grid. For each occupied voxel, at most one representative pair is retained.

Imitation Policy follows the structure proposed in [6], as it enforces true control inputs at the equilibrium $\pi_\theta(x^*) = u^*$ and satisfies the control input constraints $\underline{u} \leq \pi_\theta(x) \leq \bar{u}$ by construction. The neural policy π_θ is visualized in Fig. 6.1 and formulated as

$$\pi_\theta(x) = \text{clamp}(h_\theta(x) - h_\theta(x^*) + u^*, \underline{u}, \bar{u}). \quad (6.2)$$

where $h_\theta : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_u}$ is an unconstrained neural network, $u^* \in \mathcal{U}$ represents the desired control action at the equilibrium x^* , and $\text{clamp}(\cdot, \bar{\cdot}, \underline{\cdot})$ acts element-wise to enforce the input constraints.

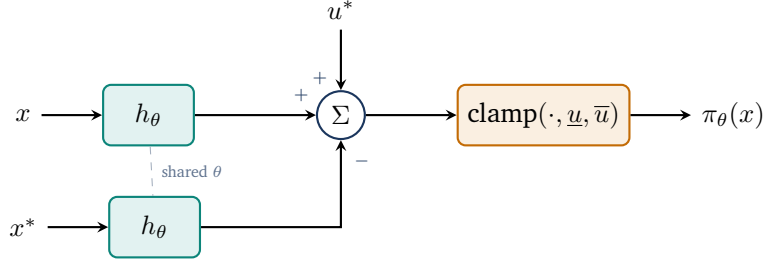


Figure 6.1: The imitation learning policy.

Imitation Objective consists of a state-weighted behavioral cloning loss and a dynamics-aware penalty. To account for different physical units and numerical scales across the state and control vectors, the loss terms are normalized using strictly positive diagonal scaling matrices

$$S_x = \text{diag}(x_{\text{scale}}) \in \mathbb{R}^{n_x \times n_x} \quad \text{and} \quad S_u = \text{diag}(u_{\text{scale}}) \in \mathbb{R}^{n_u \times n_u}. \quad (6.3)$$

Using S_x , the normalized distance of a state x from the equilibrium x^* is quantified with the L_1 -norm as

$$d(x) = \frac{1}{n_x} \|S_x^{-1}(x - x^*)\|_1. \quad (6.4)$$

To assign appropriate priority during behavioral cloning, a state-dependent weight is defined as

$$w_x(x) = w_{\min} + (1 - w_{\min}) \left(\frac{1}{1 + d(x)} \right)^\eta, \quad (6.5)$$

where $w_{\min} \in (0, 1)$ is the baseline weight for states far from the equilibrium and $\eta \geq 0$ controls the decay rate.

State-Weighted Behavioral Cloning Loss penalizes the squared normalized Euclidean distance between the neural network policy and the expert targets. For a mini-batch $\mathcal{B}_\pi \subset \mathcal{D}_\pi$ of size $M_\pi = |\mathcal{B}_\pi|$, the behavioral cloning loss is defined as

$$\mathcal{L}_{\text{BC}} = \frac{1}{M_\pi n_u} \sum_{(x,u) \in \mathcal{B}_\pi} w_x(x) \|S_u^{-1}(\pi_\theta(x) - u)\|_2^2. \quad (6.6)$$

Here, the state-dependent weight $w_x(x)$ prioritizes approximation accuracy near the equilibrium.

Dynamics-Aware Constraint Loss applies a one-step penalty for successor states that leave the admissible state set $\mathcal{X}_\pi$. For each state $x \in \mathcal{B}_\pi$, the next state is evaluated under the neural policy as $x^+ = f(x, \pi_\theta(x))$. Using the element-wise ReLU function defined in (4.2b), the loss is defined as

$$\mathcal{L}_f = \frac{1}{M_\pi n_x} \sum_{x \in \mathcal{B}_\pi} \left(\|[x^+ - \bar{x}_\pi]_+\|_1 + \|[x_\pi - x^+]_+\|_1 \right). \quad (6.7)$$

Here, a one-step evaluation outside the admissible set results in a penalty for the current state, forcing the learned policy to stay inside the admissible set.

Total Imitation Loss is the combination of the state-weighted behavioral cloning and dynamics-aware constraint losses into a single scalar objective. Scaling each term by its respective weight, ω_{BC} and ω_f , yields the overall policy loss:

$$\mathcal{L}_\pi = \omega_{\text{BC}}\mathcal{L}_{\text{BC}} + \omega_f\mathcal{L}_f. \quad (6.8)$$

6.2 Lyapunov Learning for Stability Certification

To establish formal stability guarantees for learned controllers, a neural Lyapunov candidate is trained alongside the policy. A falsification-guided framework is used to find stability violations and therefore make the Lyapunov candidate a valid certificate for stability.

6.2.1 Lyapunov Network Architecture

Following [6], the Lyapunov candidate $V_\theta(x)$ is parameterized to be positive definite by construction:

$$V_\theta(x) = |g_\theta(x) - g_\theta(x^*)| + \left\| (\varepsilon I + R_\theta^\top R_\theta)(x - x^*) \right\|_1, \quad (6.9)$$

where $g_\theta : \mathbb{R}^n \rightarrow \mathbb{R}$ is a neural network, $x \in \mathbb{R}^n$ is the current state, $x^* \in \mathbb{R}^n$ is the equilibrium, $\varepsilon > 0$ is a small regularization constant, and $R_\theta \in \mathbb{R}^{n \times n}$ is a learnable matrix. While the first term is only positive semi-definite and may vanish away from the equilibrium, the second term guarantees strict positive definiteness $\forall x \in \mathcal{X} \setminus \{x^*\}$, as the matrix $(\varepsilon I + R_\theta^\top R_\theta)$ is strictly positive definite by construction for any $\varepsilon > 0$. The complete architectural layout of $V_\theta(x)$ is illustrated in Fig. 6.2.

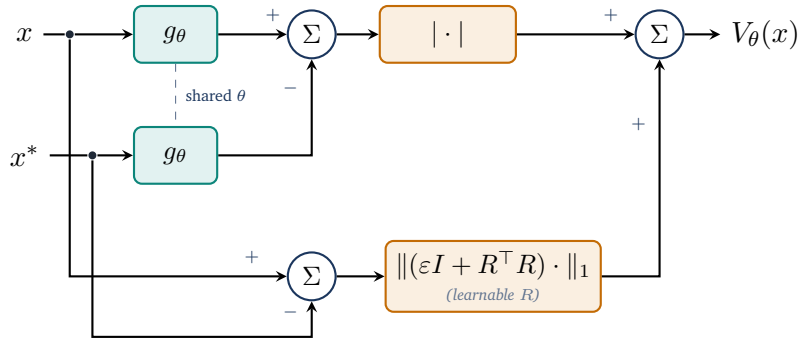


Figure 6.2: The neural Lyapunov architecture.

6.2.2 Loss Formulation

add more explanations

To ensure that the Lyapunov candidate described in (6.9) can be certified for stability, the loss function must comprise multiple components. It utilizes the previously introduced ReLU function $[\cdot]_+$ for one-sided penalties, alongside a weighted mean to aggregate the sample losses. Central to the following loss terms is the estimation of the region of attraction (ROA). Throughout the training process, a

target level ρ is considered, leading to the corresponding ρ -sublevel set within the training domain $\mathcal{X}_V := \{x \in \mathcal{X}_\pi \mid \underline{x}_V \leq x \leq \bar{x}_V\}$, defined as

$$\mathcal{V}_\rho := \{x \in \mathcal{X}_V \mid V_\theta(x) \leq \rho\}. \quad (6.10)$$

To prevent this certified region from trivially collapsing towards the origin during optimization, and to ensure numerical stability during normalizations, ρ is strictly constrained by a lower bound $\rho_{\min} > 0$. Furthermore, the successor states $x_V^+ = f(x, \pi_V(x))$ needed during loss computation are evaluated on-the-fly using the system dynamics and the policy. The latter may also be trained in the co-training setup and is thus referred to as π_V hereafter.

Relative Lyapunov Decrease Loss penalizes trajectories along which the Lyapunov candidate fails to decay. For each sample, the decrease violation is defined as

$$e_V = (1 - \kappa)V_\theta(x) - V_\theta(x_V^+), \quad (6.11)$$

where $\kappa \in (0, 1)$ is a hyperparameter that controls the desired rate of decrease. To balance the optimization scale across different magnitudes of V_θ , we normalize this violation to obtain the relative decrease loss:

$$\mathcal{L}_{\text{dec}} = \left[\frac{e_V}{\max(|V_\theta(x)|, \varepsilon_{\text{rel}})} \right]_+, \quad (6.12)$$

where $\varepsilon_{\text{rel}} > 0$ prevents division by zero. Note that the decrease normalization can be omitted depending on the training configuration.

Relative State Invariance Loss penalizes successor states leaving the Lyapunov training domain $\mathcal{X}_V$. To evaluate the relative one-step state-constraint violation, the scaling matrix $S_{\text{range}} = \text{diag}(\bar{x}_V - \underline{x}_V)$ is utilized. The loss per sample for the next state x_V^+ is computed as

$$\mathcal{L}_{\text{inv}} = \left\| S_{\text{range}}^{-1} [x_V^+ - \bar{x}_V]_+ \right\|_1 + \left\| S_{\text{range}}^{-1} [\underline{x}_V - x_V^+]_+ \right\|_1. \quad (6.13)$$

Combined ρ -Gated Condition Loss is the sample-wise ρ -gated combination of the relative Lyapunov decrease loss from (6.12) and the relative state invariance loss from (6.13),

$$\mathcal{L}_{\text{cond}} = \mathcal{L}_{\text{dec}} + \omega_{\text{inv}} \cdot \mathcal{L}_{\text{inv}}, \quad (6.14)$$

where ω_{inv} is a hyperparameter balancing the relative importance of the two loss components. However, to apply the ρ -gate appropriately, $\mathcal{L}_{\text{cond}}$ is weighted for each sample using the soft sublevel weight

$$w_\rho = \sigma \left(s \cdot \frac{\rho - V_\theta(x)}{\max(\rho, \varepsilon_{\text{rel}})} \right), \quad (6.15)$$

where $s > 0$ denotes the gate sharpness. This smooth formulation ensures that the weights inside $\mathcal{V}_\rho$ are ≈ 1 , whereas the weights for states far outside of ρ approach zero. Evaluating these terms on a mini-batch $\mathcal{B}_\rho$ drawn from the combined state pools $\mathcal{S} \cup \mathcal{C}$ (details in Section 6.2.3), the weighted mean of the combined condition loss results in the final loss formulation:

$$\mathcal{L}_{\rho\text{-gated}} = \frac{\sum_{\mathcal{B}_\rho} w_\rho \mathcal{L}_{\text{cond}}}{\sum_{\mathcal{B}_\rho} w_\rho}. \quad (6.16)$$

The complete data flow, from evaluating the batch constraints to computing this final gated loss, is summarized in Fig. 6.3.

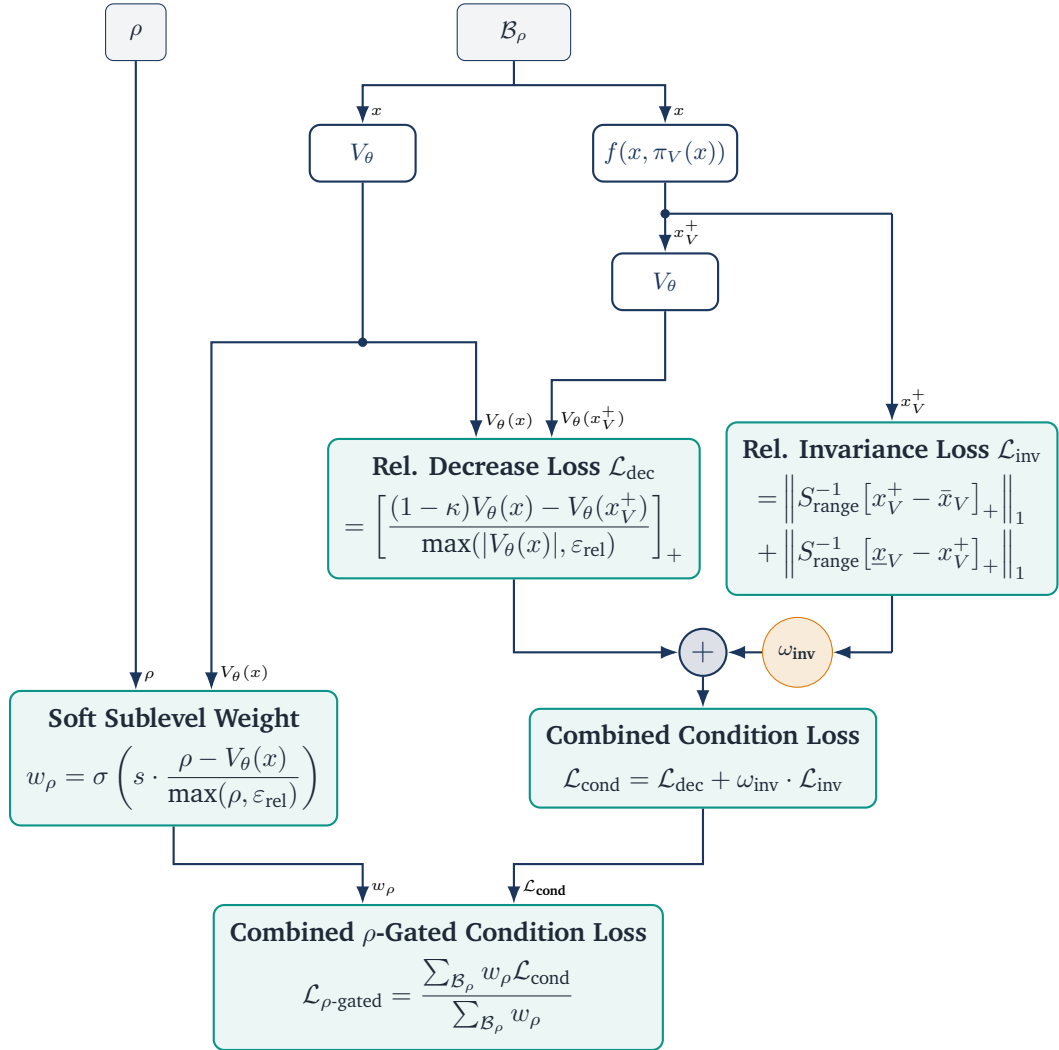


Figure 6.3: Combined gated condition loss $\mathcal{L}_{\rho\text{-gated}}$

ROA Loss tries to enlarge the region of attraction by uniformly drawing a batch $\mathcal{B}_{\text{ROA}}$ from $\mathcal{X}_V \setminus \xi \mathcal{X}_V$, where $\xi \in [0, 1)$ is a scaling factor defining the inner boundary of the shell, as shown in Fig. 6.4. The loss is then evaluated over these candidates using [6]

$$\mathcal{L}_{\text{ROA}} = \frac{1}{|\mathcal{B}_{\text{ROA}}|} \sum_{x \in \mathcal{B}_{\text{ROA}}} \ln \left(\left[\frac{V_\theta(x)}{\rho} - 1 \right]_+ + 1 \right). \quad (6.17)$$

Consequently, the loss function penalizes shell samples that fall outside the current ρ subsets. As the estimated region of attraction expands, fewer samples are penalized, causing the loss to decrease.

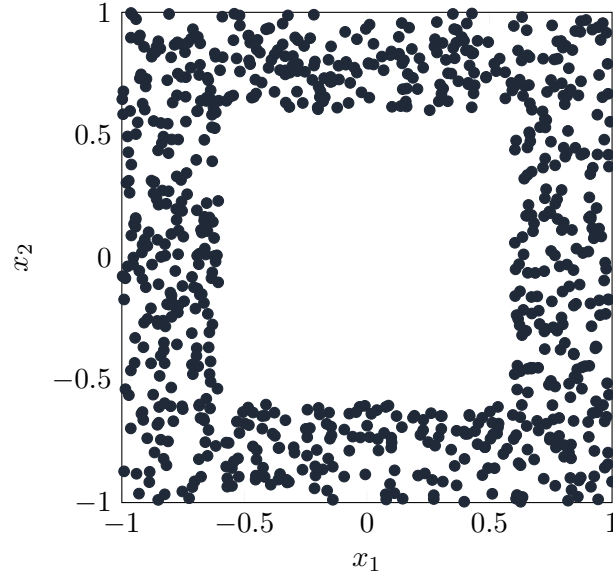


Figure 6.4: Illustration of uniformly drawn samples in the shell batch $\mathcal{B}_{\text{ROA}}$ with $\xi = 0.6$.

LIRPA-Condition Loss makes the Lyapunov function more verifier-friendly by penalizing formal violations of the Lyapunov decrease condition as defined in (6.11). These violations are identified using the LIRPA framework [7]. To reduce the computational cost during training, LIRPA is applied only to a heuristic subset of regions. Specifically, a hyper-rectangular cell is selected if at least one of its 2^{n_x} corners, or optionally its center point x_{center} , lies within the sublevel set $\mathcal{V}_\rho$ (i.e., $\min(V(x)) \leq \rho$ evaluated over these points). For each selected region, LIRPA then provides a formal lower bound $e_{V,\text{lower}}$ on the signed Lyapunov decrease $-e_V$. After evaluation, the LIRPA-condition violation for each region is computed as

$$\mathcal{M}_L = \frac{\ln([-e_{V,\text{lower}}]_+ + 1)}{\max(V_\theta(x_{\text{center}}), \varepsilon_{\text{rel}})}. \quad (6.18)$$

The final LIRPA-condition loss is then computed as the weighted mean over all evaluated regions, where the weight is defined using an exponential function of the squared Euclidean distance between the region center x_{center} and the equilibrium x^* :

$$w_L = \exp \left(-\alpha_L \cdot \|x_{\text{center}} - x^*\|_2^2 \right), \quad (6.19)$$

where $\alpha_L > 0$ is a hyperparameter controlling the decay rate of the weight. Finally, the LIRPA-condition loss over N_L regions is computed as

$$\mathcal{L}_L = \frac{\sum_{i=1}^{N_L} w_L^{(i)} \cdot \mathcal{M}_L^{(i)}}{\sum_{i=1}^{N_L} w_L^{(i)}}. \quad (6.20)$$

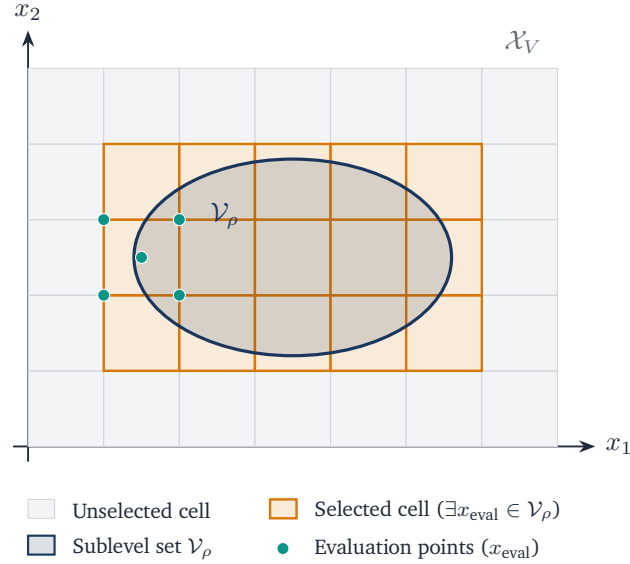


Figure 6.5: Lirpa regions selection with $\mathcal{V}_\rho$.

Lyapunov Scaling Loss prevents numerical collapse to zero by penalizing deviations from the initial average magnitude over a quasi-Monte Carlo batch $\mathcal{B}_V \subset \mathcal{X}_V$, sampled via a low-discrepancy Sobol sequence [8], as illustrated in Fig. 6.6. The logarithm of the mean Lyapunov value over this batch is defined as

$$\ell_V = \ln \left(\frac{1}{|\mathcal{B}_V|} \sum_{x \in \mathcal{B}_V} V_\theta(x) \right), \quad (6.21)$$

leading to the scaling loss formulation

$$\mathcal{L}_V = (\ell_{V,0} - \ell_V)^2. \quad (6.22)$$

The reference value $\ell_{V,0}$ is determined once using the initial Lyapunov candidate prior to training. To prevent the Lyapunov candidate from overfitting to a static point cloud, a new batch $\mathcal{B}_V$ is resampled from $\mathcal{X}_V$ with a specified probability at each optimization step.

Policy Scaling Loss encourages the learned policy π_V to remain close to the output of the original imitation controller π , which is based on the MPC teacher. A quasi-Monte Carlo batch $\mathcal{B}_{S\pi} \subset \mathcal{X}_V$ is sampled using a low-discrepancy Sobol sequence [8]. To prevent overfitting to a static set of states,

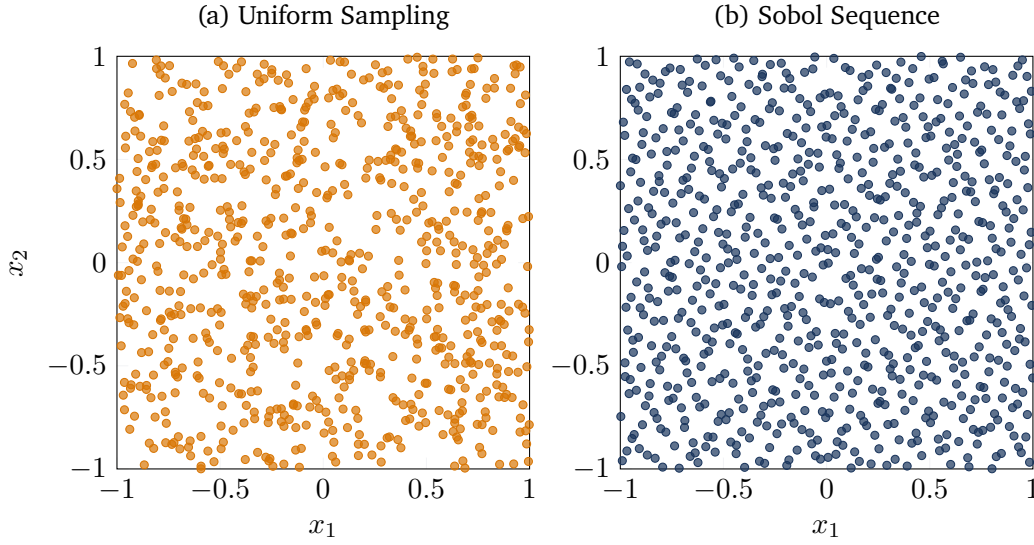


Figure 6.6: Comparisson of Sobol and uniform sampling in two dimensions.

this batch also is resampled from $\mathcal{X}_V$ after a specified number of optimization steps. The normalized deviation between the learned policy and the imitation controller over this batch is formulated as

$$\mathcal{L}_{S\pi} = \frac{\sum_{x \in \mathcal{B}_{S\pi}} \|\pi_V(x) - \pi(x)\|_2^2}{\sum_{x \in \mathcal{B}_{S\pi}} \|\pi(x)\|_2^2} \quad (6.23)$$

R -Matrix Loss regularizes the learnable matrix R_θ by penalizing deviations of its Frobenius norm from its initial value $\ell_{R,0}$:

$$\mathcal{L}_R = (\ell_{R,0} - \|R\|_F)^2. \quad (6.24)$$

However, $\ell_{R,0} = \|R_\theta\|_F$ is automatically computed, based on the initial $R_{\theta,0}$ matrix.

Overall Lyapunov Loss represents the combined objective function, summing all previously defined weighted components, given by

$$\mathcal{L}_{\text{total}} = \sum_m \omega_m \mathcal{L}_m \quad (6.25)$$

where the weights $\omega_m > 0$, with $m \in \{\rho\text{-gated}, \text{ROA}, \text{L}, \text{V}, \text{S}\pi, \text{R}\}$ are hyperparameters that balance the relative importance of the individual loss components. The full loss flow is pictured in Fig. 6.7

6.2.3 Falsification-Guided Training

Relying solely on uniform state sampling is not sufficient to reliably detect violations of the Lyapunov condition, especially in higher-dimensional state spaces **counterexampleRef**. To overcome this problem, falsification-guided training actively searches for adversarial counterexamples and incorporates them into the training distribution. This is made possible by an adversarial replay buffer that reconciles the exploration of the estimated attractor region with the targeted exploitation of known error modes.

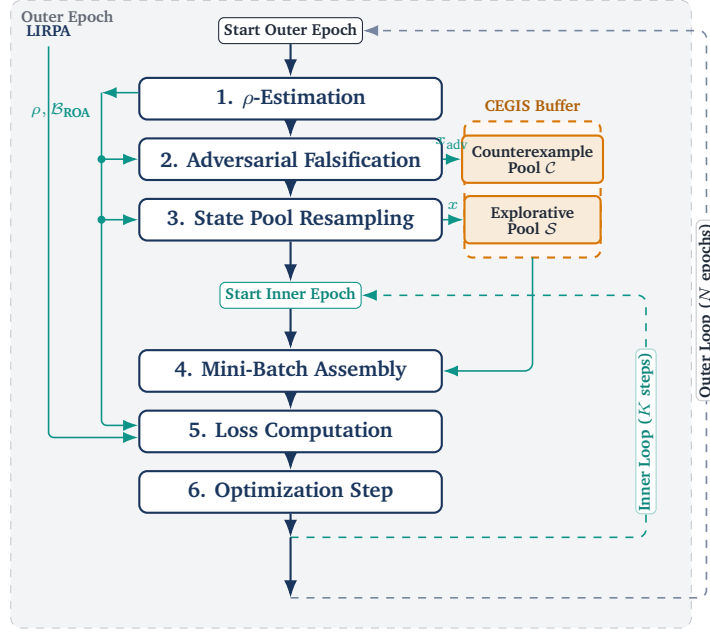


Figure 6.7: Loss flow visualization.

PGD Falsification is utilized to identify counterexamples that violate the Lyapunov conditions and iteratively inject them into the counterexample pool $\mathcal{C}$. Specifically, an L_∞ -Projected Gradient Descent (PGD) is employed to find adversarial states $x_{\text{adv}} \in \mathcal{X}_V$ that maximize the condition violation defined in (6.14) and (6.15). The per-sample PGD objective is defined as

$$J_{\text{PGD}} = -w_\rho \mathcal{L}_{\text{cond}}. \quad (6.26)$$

The adversarial search is initialized using the explorative pool $\mathcal{S}$, with up to 25% of the batch supplemented by existing counterexamples to encourage local refinement. The algorithm minimizes J_{PGD} by iteratively updating the input states along the sign of their local gradient with a constant step size α , followed by a projection back onto the training bounds $\mathcal{X}_V$. To retain the strongest violations and mitigate the effects of overshooting, the state yielding the minimum objective value across all K iterations is preserved for each candidate. Ultimately, only states strictly within $\mathcal{V}_\rho$ that exceed the violation tolerance and lie outside the origin exclusion region are retained as counterexamples.

Adversarial Replay Buffer manages the sample distribution for the ρ -gated condition loss $\mathcal{L}_{\rho\text{-gated}}$. It maintains two distinct state pools: an explorative pool $\mathcal{S}$ and a counterexample pool $\mathcal{C}$.

As outlined in Algorithm 1, the explorative state pool $\mathcal{S}$ is repopulated at each outer epoch to balance global exploration with local refinement. First, candidate states $\mathcal{B}_{\text{ROA}}$ are oversampled uniformly from the training domain $\mathcal{X}_V$. From these candidates, N_{ROA} states are probabilistically sampled based on a sigmoid weighting function centered around an expanded sublevel boundary $m \cdot \rho$ (with margin $m > 1$). This weighting smoothly penalizes states further outside the estimated region of attraction, significantly reducing their likelihood of being selected.

The counterexample pool $\mathcal{C}$ stores the critical violations identified during the PGD falsification step. A age-tracking mechanism limits sample lifespan, preventing overfitting outdated violations. Each

Algorithm 1: Probabilistic Explorative State Pool Resampling

Input: Lyapunov function V_θ , current ROA estimate ρ , margin m , sharpness s , target size N_{ROA}

Output: Updated state pool $\mathcal{S}$ of size N_{ROA}

- 1 Generate $4N_{\text{ROA}}$ candidates: $\mathcal{B}_{\text{ROA}} \sim \mathcal{U}(\mathcal{X}_V)$;
 - 2 For all $x \in \mathcal{B}_{\text{ROA}}$, assign weight $w(x) \leftarrow \sigma\left(s \frac{m \cdot \rho - V_\theta(x)}{\max(\rho, 10^{-9})}\right) + \varepsilon$;
 - 3 $\mathcal{S} \leftarrow$ Sample N_{ROA} states from $\mathcal{B}_{\text{ROA}}$ with probability $P(x) \propto w(x)$;
-

injected counterexample is assigned an initial age $\tau = 0$, which is incremented after every subsequent falsification phase. Once a counterexample reaches the maximum age τ_{max} , it is removed from the pool. Consequently, the condition loss continuously penalizes recent violations while discarding resolved edge cases as the Lyapunov candidate evolves.

6.2.4 ρ -Sublevel Estimation

explain why exactly we need it

To compute the loss, particularly the ρ -gated condition loss in (6.16), the target level ρ must be continuously estimated during training. Since the goal is to find the largest level set of the Lyapunov candidate $V_\theta(x)$ that lies entirely within the defined training bounds $\mathcal{X}_V$, a level set can only escape the safe region by intersecting its boundary $\partial\mathcal{X}_V$. Therefore, the maximum certifiable ρ is mathematically strictly determined by the minimum value of the Lyapunov candidate at its boundary. According to the formulation proposed by [6], it is sufficient to evaluate $V_\theta(x)$ only on the boundary $\partial\mathcal{X}_V$ rather than sampling the entire state space. However, since the boundary is a high-dimensional surface and the neural network parameterized in $V_\theta(x)$ forms a highly non-convex landscape, simple uniform sampling on the boundary is insufficient to find the true minimum. Therefore, a Projected Gradient Descent (PGD) approach is employed to actively search for the worst-case boundary points. Starting from uniformly sampled points $\mathcal{B}_\partial \sim \mathcal{U}(\partial\mathcal{X}_V)$, PGD iteratively minimizes $V_\theta(x)$ by taking steps along the negative gradient and projecting the updated states strictly back onto the boundary surfaces. To actively encourage the region of attraction to expand over the course of the co-training process, the formally certifiable limit is intentionally overestimated, as suggested in [6]. Specifically, a target ρ is computed by applying a growth factor $\gamma > 1$ to the boundary minimum found via PGD:

$$\tilde{\rho} = \max\left(\rho_{\min}, \gamma \cdot \min_{x \in \mathcal{B}_\partial} V_\theta(x)\right). \quad (6.27)$$

By targeting a value for $\tilde{\rho}$ that is slightly above the currently safe maximum, the formulation deliberately leads to small violations near the boundary. These violations are then captured by the aforementioned adversarial replay buffer, effectively forcing the optimization to continuously expand the sublevel set outward. Finally, to ensure numerical stability and prevent irregular jumps in the loss landscape caused by fluctuating local minima across different batches, the ρ value actually used for gating is updated using an exponential moving average (EMA):

$$\rho = \beta\rho + (1 - \beta)\tilde{\rho}, \quad (6.28)$$

where $\beta \in [0, 1)$ is the decay rate.

6.3 Formal Verification Methodology

This section adapts the search algorithm for the maximal certifiable sublevel set $\mathcal{V}_\rho$ proposed in [6]. However, iteration limits for scaling and bisection are integrated into Algorithm 2 to reduce computational overhead during evaluation.

change $\Phi(\rho)$ to a more general formulation that evaluates the whole set of x bounds.

To formally certify the Lyapunov candidate, the $\alpha\beta$ -CROWN verifier evaluates a logical formulation $\Phi(x; \rho)$ over a bounded certification domain $\mathcal{B}_C := \{x \in \mathbb{R}^n \mid \underline{x}_C \leq x \leq \bar{x}_C\} \subseteq \mathcal{B}_V$. The goal is to verify that all states in $\mathcal{V}_\rho \cap \mathcal{B}_C$ satisfy the required stability conditions. This implication is rewritten into a disjunctive form that is directly supported by the verifier: a state must either lie outside $\mathcal{V}_\rho$ or satisfy all stability conditions.

The individual logical terms from which this specification is composed are defined as follows:

$$\Phi_{\text{sub}} \equiv V_\theta(x) > \rho, \quad (6.29a)$$

$$\Phi_{\text{pos}} \equiv V_\theta(x) > 0, \quad (6.29b)$$

$$\Phi_{\text{inv}} \equiv \underline{x}_C \leq x^+ \leq \bar{x}_C, \quad (6.29c)$$

$$\Phi_{\text{dec}} \equiv (1 - \kappa)V_\theta(x) - V_\theta(x^+) > 0, \quad (6.29d)$$

where $x^+ = f(x, \pi_\theta(x))$ denotes the discrete one-step state transition. Here, (6.29a) determines whether x lies outside the target sublevel set $\mathcal{V}_\rho$. For states within $\mathcal{V}_\rho$, (6.29b) enforces strict positive definiteness away from x^* , (6.29c) guarantees forward invariance within $\mathcal{B}_C$, and (6.29d) ensures the strict Lyapunov decrease property.

Finally, combining the sublevel exclusion with the joint stability conditions yields the total specification evaluated by the verifier:

$$\Phi(x; \rho) \equiv \Phi_{\text{sub}} \vee (\Phi_{\text{pos}} \wedge \Phi_{\text{inv}} \wedge \Phi_{\text{dec}}). \quad (6.30)$$

The equilibrium x^* must be excluded from the certification domain $\mathcal{B}_C$. By construction, $V_\theta(x^*) = 0$ and $x^+ = x^*$ hold at the equilibrium, causing the strict positivity (Φ_{pos}) and strict decrease (Φ_{dec}) conditions to evaluate to false at x^* . Since these conditions hold only for $x \neq x^*$ and positive definiteness is enforced by construction, formal verification is performed over the punctured domain $\mathcal{B}_C \setminus \mathcal{B}_{\text{hole}}$. Here, $\mathcal{B}_{\text{hole}}$ denotes a small hyper-rectangular region centered at x^* . The punctured certification domain is thus split into multiple subregions, as shown in Fig. 6.8.

Algorithm 2: Certified Value Search via Scaling and Bisection

Input: $\rho_0, \rho_{\min}$, evaluator Φ , scale $s > 1$, tol ε , limits $N_{\text{scale}}, N_{\text{bisection}}$

Output: Max certified value ρ^*

```
1  $\rho \leftarrow \max(\rho_0, \rho_{\min})$ ;
  // Phase 1: Scaling
2 if  $\Phi(\rho)$  then
3   for  $k \leftarrow 1$  to  $N_{\text{scale}}$  do
4     if  $\neg\Phi(\rho)$  then
5       break;
6     end
7      $\rho_{\text{lo}} \leftarrow \rho$ ;  $\rho \leftarrow \rho \cdot s$ ;
8   end
9    $\rho_{\text{up}} \leftarrow \rho$ ;
10 else
11   for  $k \leftarrow 1$  to  $N_{\text{scale}}$  do
12     if  $\Phi(\rho) \vee \rho = \rho_{\min}$  then
13       break;
14     end
15      $\rho_{\text{up}} \leftarrow \rho$ ;  $\rho \leftarrow \max(\rho_{\min}, \rho/s)$ ;
16   end
17   if  $\neg\Phi(\rho)$  then
18     return 0 ; // No valid value  $\geq \rho_{\min}$ 
19   end
20    $\rho_{\text{lo}} \leftarrow \rho$ ;
21 end

  // Phase 2: Bisection
22 for  $k \leftarrow 1$  to  $N_{\text{bisection}}$  do
23   if  $\rho_{\text{up}} - \rho_{\text{lo}} \leq \varepsilon$  then
24     break;
25   end
26    $\rho_{\text{mid}} \leftarrow \frac{1}{2}(\rho_{\text{lo}} + \rho_{\text{up}})$ ;
27   if  $\Phi(\rho_{\text{mid}})$  then
28      $\rho_{\text{lo}} \leftarrow \rho_{\text{mid}}$ ;
29   else
30      $\rho_{\text{up}} \leftarrow \rho_{\text{mid}}$ ;
31   end
32 end
33 return  $\rho_{\text{lo}}$ ;
```

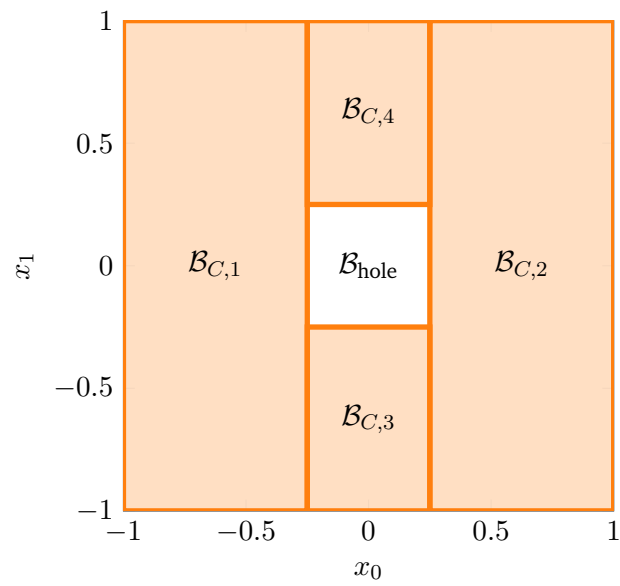


Figure 6.8: 2D Hole exclusion example with $B_{\text{hole}} := \{x \in \mathbb{R}^2 \mid \|x\|_\infty < 0.25\}$.

7 Experimental Evaluation and Certification Results

7.1 System Modeling and Problem Formulation

For our experimental setup, two benchmark systems are selected: the double integrator and the cartpole. These systems provide a good balance between simplicity and complexity allowing us to certify the stability of the closed-loop system.

7.1.1 Double Integrator

The double integrator is a simple second-order linear system; therefore, it serves as the ideal baseline system in the learning and certification pipeline. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.1$, leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\text{DI},d}(x_k, u_k) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ \Delta t \end{bmatrix} u_k \quad (7.1)$$

where $x_k = [p_k \ v_k]^\top$ is the state vector containing the position and velocity of the system, respectively, and u_k denotes acceleration.

The *MPC setup* is formulated according to the problem in (3.4) subject to the double-integrator dynamics in (7.1). A prediction horizon of $N = 20$ steps is chosen, and the states and inputs are penalized using the weighting matrices $Q = \text{diag}(15, 1)$ and $R = 0.1$.

7.1.2 Cartpole

The inverted pendulum on a cart, commonly referred to as the cartpole system, is used as a nonlinear benchmark system in this work. There are four states in the system: the cart position p , the cart velocity v , the pendulum angle θ , and the pendulum angular velocity ω . Here, the state vector is defined as $x = [p \ v \ \theta \ \omega]^\top$, and the input u is the force F applied to the cart. This leads to the following continuous-time dynamics of the cartpole system:

$$f_{\text{CP}}(x, u) = \begin{bmatrix} \dot{p} \\ \dot{v} \\ \dot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} v \\ \frac{F + ml\omega^2 \sin(\theta) - mg \sin(\theta) \cos(\theta)}{M + m - m \cos^2(\theta)} \\ \omega \\ \frac{-F \cos(\theta) - ml\omega^2 \sin(\theta) \cos(\theta) + (M + m)g \sin(\theta)}{l(M + m - m \cos^2(\theta))} \end{bmatrix} \quad (7.2)$$

where $M = 1$ kg is the mass of the cart, $m = 0.1$ kg is the mass of the pendulum, $l = 0.8$ m is the length of the pendulum, and $g = 9.81$ m/s² is the acceleration due to gravity. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.05$ s, leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\text{CP},d}(x_k, u_k) = x_k + \Delta t f_{\text{CP}}(x_k, u_k) \quad (7.3)$$

For the nonlinear benchmark, a prediction horizon of $N = 40$ stages is chosen. The system states and control inputs are penalized within the optimization framework of (3.4) using the weighting matrices $Q = \text{diag}(10, 1, 10, 0.01)$ and $R = 0.5$, respectively, subject to the dynamics defined in (7.3).

7.2 Linear System Results: Double Integrator

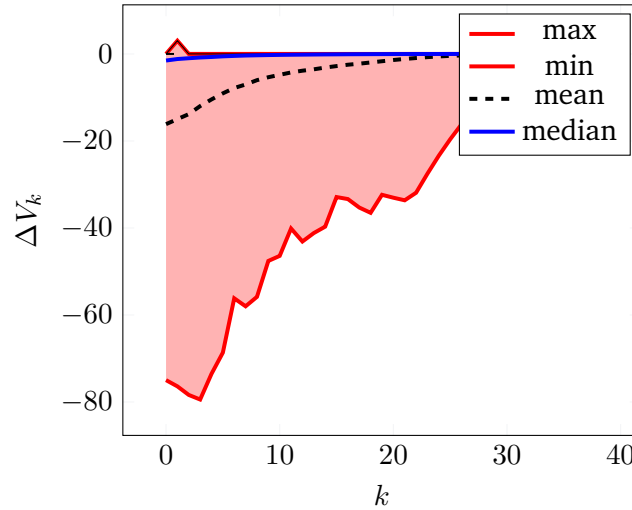


Figure 7.1: Double integrator cost descent. $\Delta V = V_N(x_{k+1}) - V_N(x_k)$

7.3 Nonlinear System Results: Cartpole

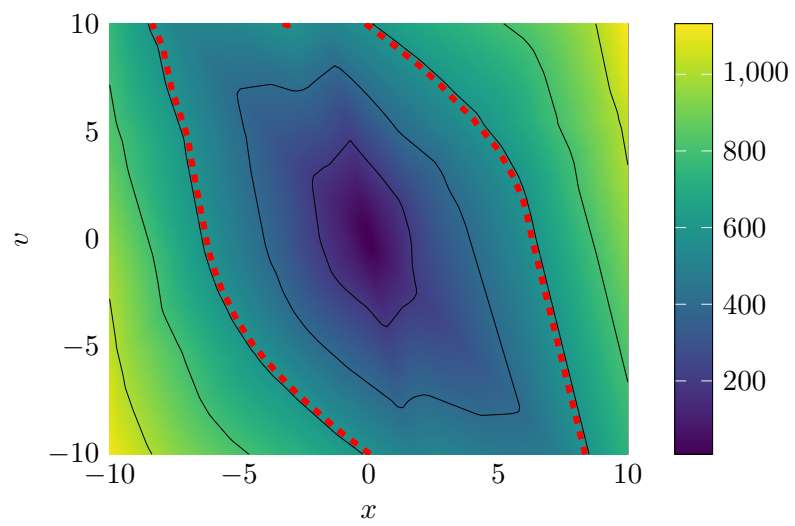


Figure 7.2: Double integrator Lyapunov landscape and ROA: value



8 Discussion

8.1 Qualitative Discussion of Results

8.2 Challenges with Nonlinear Dynamics

8.3 Scalability and Limitations



9 Conclusion



A Appendix

Bibliography

- [1] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Chichester, UK: Wiley, 2016, 1 p., ISBN: 978-1-119-12150-3 978-1-119-12151-0 978-1-119-12153-4. DOI: 10.1002/9781119121534.
- [2] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation and Design*, 2nd edition. Santa Barbara: Nob Hill Publishing, LLC, 2020, 623 pp., ISBN: 978-0-9759377-5-4.
- [3] L. Grüne and J. Pannek, *Nonlinear Model Predictive Control: Theory and Algorithms* (Communications and Control Engineering), 2nd ed. 2017. Cham: Springer, 2017, 456 pp., ISBN: 978-3-319-46024-6. DOI: 10.1007/978-3-319-46024-6.
- [4] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, Second Edition. Cham: Springer International Publishing AG, 2023, 529 pp., ISBN: 978-3-031-29642-0 978-3-031-29641-3.
- [5] S. Kollmannsberger, D. D'Angella, M. Jokeit, and L. Herrmann, *Deep Learning in Computational Mechanics: An Introductory Course* (Studies in Computational Intelligence volume 977). Cham: Springer, 2021, 104 pp., ISBN: 978-3-030-76587-3 978-3-030-76589-7.
- [6] L. Yang, H. Dai, Z. Shi, C.-J. Hsieh, R. Tedrake, and H. Zhang. "Lyapunov-stable Neural Control for State and Output Feedback: A Novel Formulation". arXiv: 2404.07956 [cs.LG]. (Jun. 5, 2024), [Online]. Available: <http://arxiv.org/abs/2404.07956> (visited on 06/24/2026), pre-published.
- [7] K. Xu, H. Zhang, S. Wang, *et al.* "Fast and Complete: Enabling Complete Neural Network Verification with Rapid and Massively Parallel Incomplete Verifiers". arXiv: 2011.13824 [cs.AI]. (Mar. 16, 2021), [Online]. Available: <http://arxiv.org/abs/2011.13824> (visited on 06/24/2026), pre-published.
- [8] S. Joe and F. Y. Kuo, "Constructing Sobol Sequences with Better Two-Dimensional Projections", *SIAM Journal on Scientific Computing*, vol. 30, no. 5, pp. 2635–2654, Jan. 2008, ISSN: 1064-8275, 1095-7197. DOI: 10.1137/070709359. [Online]. Available: <http://epubs.siam.org/doi/10.1137/070709359> (visited on 08/03/2026).